

08

Módulo 08 - Tipos de aprendizagem - aprendizagem por reforço

Fundamentos de Inteligência Artificial (IA)

Resumo do Módulo

- ◆ O Processo de Aprender Através de Recompensas e Penalidades.
- ◆ Exemplos e Aplicações em Jogos, Robótica e Otimização.
- ◆ Lab

Machine Learning

◆ Supervisionada (Supervised):

- Regressão (Regression): **Linear Simples** • Linear Múltipla • Polinomial • Lasso • Ridge
- Classificação (Classification): **Regressão Logística** • KNN • SVM • Kernel SVM • Naïve Bayes
- Séries Temporais (Time Series): **ARIMA** • SARIMA • LSTM • Prophet • Suavização Exponencial
- Métodos de Conjunto (Ensemble Methods): **Árvore de Decisão** • Random Forest • XGBoost • CatBoost
- Aprendizagem Profunda (Deep Learning): **Redes Neurais Artificiais** • Convolucionais • Recorrentes • **Transformers + Atenção**

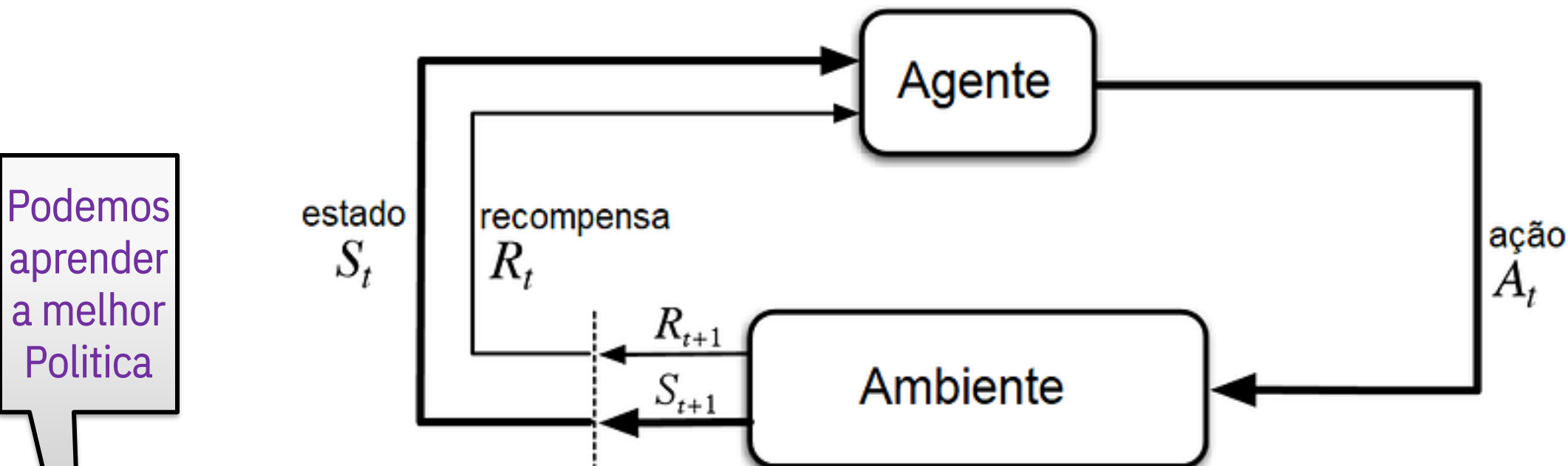
◆ Não Supervisionada (Unsupervised):

- Clustering (Agrupamento): **K-means** • Hierárquico • GMM
- Redução de Dimensionalidade (Dimensionality Reduction): **PCA** • Kernel PCA • LDA

◆ Reforço (Reinforcement):

- Algoritmos de Reforço (Reinforcement Algorithms): **Limite de Confiança Superior** • Amostragem de Thompson • **Q-Learning** • **Deep Q-Learning** • PPO
- Algoritmos Genéticos (Genetic Algorithms): **Algoritmo Genético Clássico** • NSGA-II • Algoritmos Meméticos

O Processo de Aprender Através de Recompensas e Penalidades



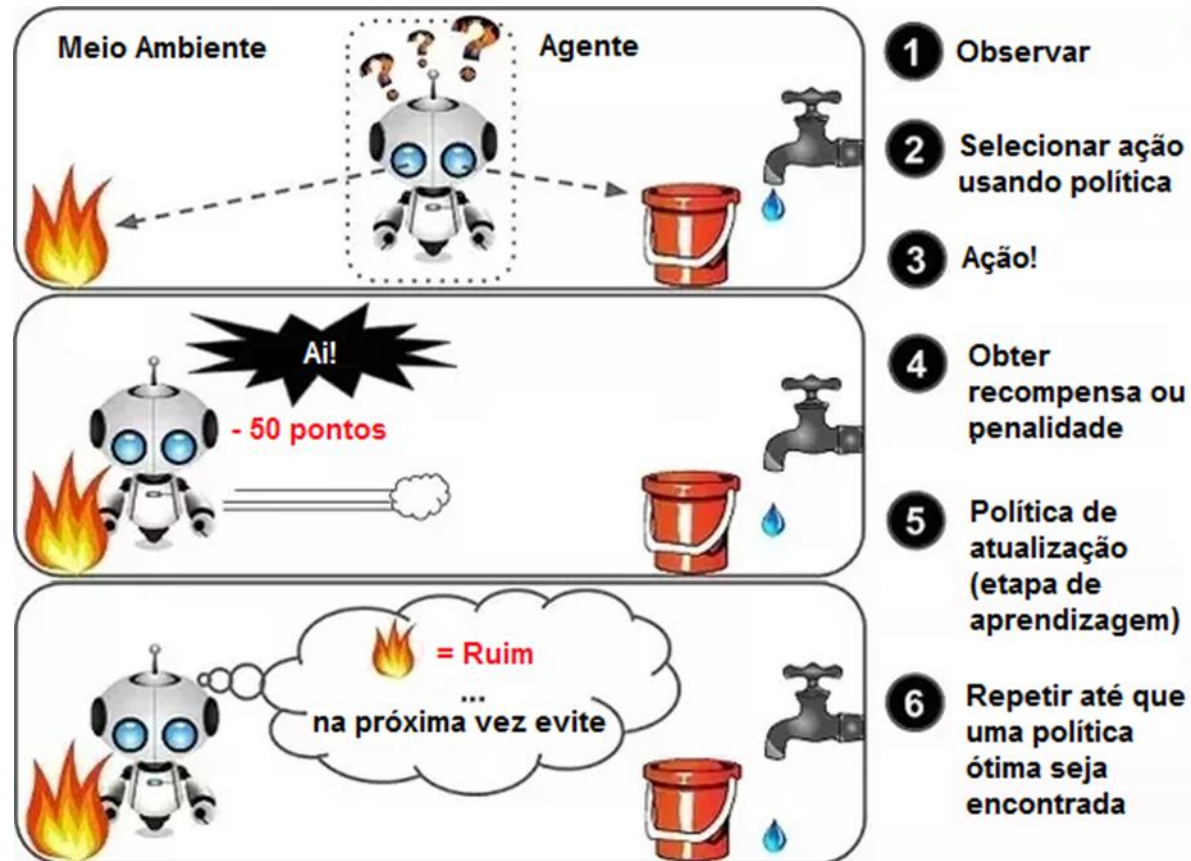
O **Agente** interage com o **Ambiente** por meio de uma **Ação** escolhida por uma **Política**, e recebe uma **Recompensa** (positiva ou negativa – **Penalidade**) e um novo **Estado** para escolher a próxima **Ação**.

Podemos aprender o Valor do Estado

Podemos aprender a Qualidade da Ação

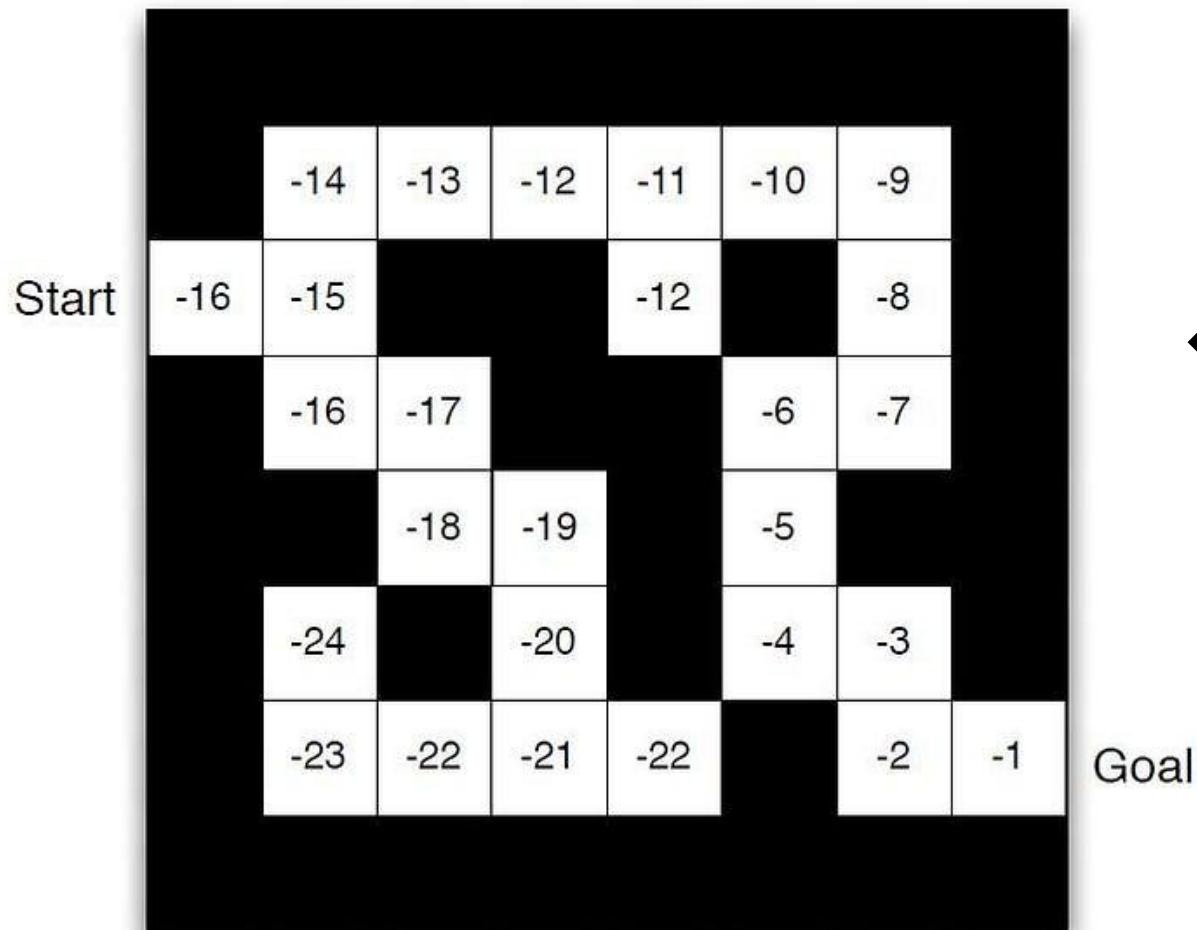
A **Política** refere-se ao processo de decisão do Agente.

♦ é o **algoritmo** que ele usa para escolher a melhor ação para cada estado.



Valor de um Estado (V)

O **Valor de um Estado** específico, ou **Função de Valor**, é denominado $V(s)$, e consiste no retorno esperado a partir daquele determinado estado.



- ◆ se o **Valor de S1** é maior que o **Valor de S2**, nós temos que tentar chegar a **S1** em vez de **S2**

Qualidade de uma Ação (Q)

Q de uma Ação a é o **retorno provável (estatístico)** ao se tomar aquela ação.

+ um valor de Q alto indica uma ação favorável (probabilidade de sucesso alta)

- um valor de Q baixo indica uma ação desfavorável (probabilidade baixa)

$$\text{New } Q(s, a) = Q(s, a) + \alpha [R(s, a) + \gamma \max_{a'} Q'(s', a') - Q(s, a)]$$

newQ (s, a) é o novo valor que estamos a calcular para o par estado-ação entrar na **Q_Table**

Q (s, a) é o estado atual

Alfa é a taxa de aprendizagem

R (s, a) é a recompensa por aquele par estado-ação

Gama é a taxa de desconto

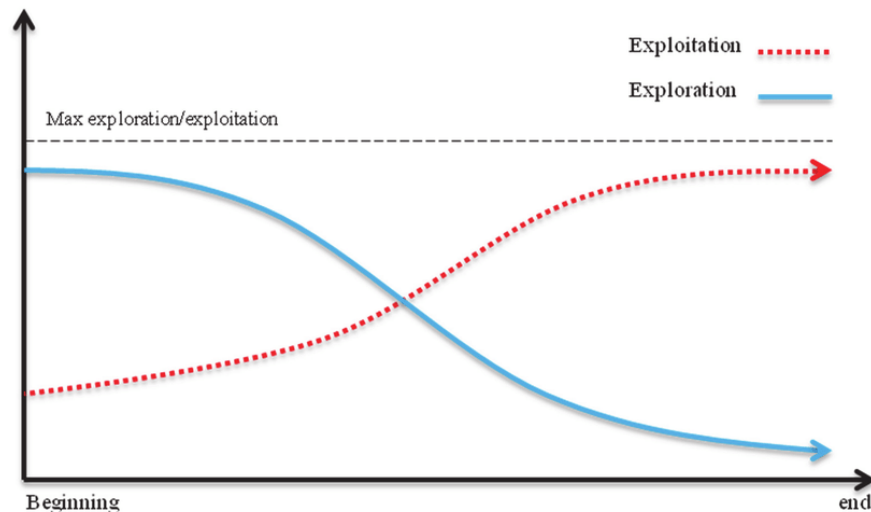
maxQ (s', a') é a recompensa futura máxima esperada
dada ao novo estado

ESTADOS	AÇÕES (Probabilidade de Sucesso)			
	CIMA	BAIXO	ESQUERDA	DIREITA
...	...	...	...	...
.... .	0.27	0.16	0.07	0.50
.	0.01	0.14	0.60	0.25
... ..	0.46	0.14	0.00	0.40
... ..	0.01	0.44	0.22	0.33
...	...	...	...	...

Q-Table já treinada (a melhor opção a **azul**)

Dilema exploração e exploração

- ♦ **Exploração (exploration)**: O agente realiza ações para **adquirir mais conhecimento** sobre o ambiente, sacrificando recompensas a curto prazo para melhorar o desempenho futuro.
- ♦ **Exploração (exploitation)** [Aproveitar]: O agente seleciona a ação que oferece a **maior recompensa** imediata com base no **conhecimento atual**, visando maximizar o ganho a curto prazo.
 - É uma abordagem Greedy ("gulosa") que visa maximizar o ganho a curto prazo.



Multi-armed bandit problema (ou k-armed-bandit problem)

- ♦ **Multi-armed bandit** = Como escolher entre várias opções quando não sabe qual é a melhor?

Exemplo prático:

- ♦ Tem 5 anúncios diferentes para mostrar no seu site
- ♦ Cada anúncio tem uma taxa de clique diferente (desconhecida)
- ♦ Precisa-se decidir entre:
 1. Testar anúncios novos (explorar)
 2. Usar os que já sabemos que funcionam bem (aproveitar)

Comparar algoritmos de reforço

Algoritmo	Tipo	Prós	Contras
UCB	Exploração vs. Exploração	Simples; eficaz em problemas fixos	Não lida bem com ambientes complexos
Thompson Sampling	Exploração vs. Exploração	Adapta-se à incerteza; bom desempenho	Requer distribuição de recompensas
Q-Learning	Baseado em Valores	Funciona em problemas de múltiplos estados	Lento; requer muitos episódios
Deep Q-Learning	Deep Learning	Alta dimensionalidade; lida com complexidade	Complexo; requer recursos significativos
PPO	Baseado em Política	Estável e eficiente	Complexo de implementar

Limite de Confiança Superior (UCB)

Resolver o dilema de exploração vs. exploração em ambientes com múltiplas opções (ex.: multi-armed bandit).

```
# pip install mabwiser
```

```
from mabwiser.mab import MAB, UCB
import numpy as np
```

```
# Definindo as opções (braços) e recompensas
```

```
opcoes = [0, 1, 2] # Três opções (exemplo: três anúncios diferentes)
```

```
recompensas = np.random.randint(1, 10, size=(10, 3)) # Recompensas aleatórias para cada rodada
```

```
# Criando o modelo UCB
```

```
mab = MAB(arms=opcoes, learner=UCB)
```

```
mab.fit(decisions=[], rewards=[]) # Inicialmente sem decisões e recompensas
```

```
# Realizando escolhas usando UCB
```

```
for rodada in range(10):
```

```
    opcao = mab.predict() # Escolha uma opção com base em UCB
```

```
    recompensa = recompensas[rodada, opcao] # Obtendo a recompensa simulada para a opção escolhida
```

```
    mab.partial_fit([opcao], [recompensa]) # Atualiza o modelo com a escolha e recompensa
```

```
    print(f"Rodada {rodada + 1}: Opção escolhida: {opcao}, Recompensa: {recompensa}")
```

Thompson Sampling

Thompson Sampling também lida com o problema de exploração vs. exploração, mas de forma probabilística.

```
# pip install mabwiser

from mabwiser.mab import MAB, ThompsonSampling
import numpy as np

# Definindo as opções (braços) e recompensas
opcoes = [0, 1, 2] # Três opções (ex.: três anúncios diferentes)
recompensas = np.random.randint(0, 2, size=(10, 3)) # Recompensas binárias (0 ou 1)

# Criando o modelo Thompson Sampling
mab = MAB(arms=opcoes, learner=ThompsonSampling)
mab.fit(decisions=[], rewards=[]) # Inicialmente sem decisões e recompensas

# Realizando escolhas usando Thompson Sampling
for rodada in range(10):
    opcao = mab.predict() # Escolha uma opção com base no Thompson Sampling
    recompensa = recompensas[rodada, opcao] # Obtendo a recompensa simulada para a opção escolhida
    mab.partial_fit([opcao], [recompensa]) # Atualiza o modelo com a escolha e recompensa
    print(f"Rodada {rodada + 1}: Opção escolhida: {opcao}, Recompensa: {recompensa}")
```

Q-Learning

Aprende a melhor política em um ambiente de múltiplos estados e ações.

- Atualiza uma **tabela de Q-valores** que representa a expectativa de recompensa para cada ação em cada estado.

```
# pip install gym gym-q-learning-agents
```

```
import gym
```

```
from gym_q_learning_agents import QLearningAgent
```

```
# Criar o ambiente e configurar o agente
```

```
env = gym.make('FrozenLake-v1', is_slippery=False)
```

```
agent = QLearningAgent(env, alpha=0.1, gamma=0.99, epsilon=0.2)
```

```
# Treinar o agente
```

```
agent.learn(total_episodes=1000)
```

```
# Testar o modelo treinado
```

```
state = env.reset()
```

```
done = False
```

```
while not done:
```

```
    action = agent.choose_action(state, explore=False)
```

```
    state, reward, done, _ = env.step(action)
```

```
    env.render()
```

Deep Q-Learning (DQN)

Deep Q-Learning (DQN) é um aprimoramento do **Q-Learning** que usa redes neurais para aproximar os Q-valores, possibilitando lidar com ambientes maiores

```
# pip install gym stable-baselines3[extra]
```

```
import gym
from stable_baselines3 import DQN
```

```
# Criar o ambiente do Gym
env = gym.make('FrozenLake-v1', is_slippery=False)
```

```
# Criar e treinar o modelo DQN (Deep Q-Learning)
# DQN é uma versão aprimorada do Q-Learning que usa redes neurais para estimar Q-valores
model = DQN('MlpPolicy', env, learning_rate=0.01, verbose=1)
model.learn(total_timesteps=10000)
```

```
# Testar o modelo treinado
obs = env.reset()
for _ in range(10):
    action, _states = model.predict(obs, deterministic=True)
    obs, reward, done, info = env.step(action)
    env.render()
    if done:
        obs = env.reset()
```

Proximal Policy Optimization (PPO)

- ♦ **PPO** otimiza diretamente a política do agente, ajustando a probabilidade de ações para maximizar a recompensa esperada, enquanto mantém a mudança dentro de limites para garantir estabilidade.

```
# pip install gym stable-baselines3[extra]
```

```
import gym
from stable_baselines3 import PPO
```

```
# Criar o ambiente do Gym
env = gym.make('CartPole-v1')
```

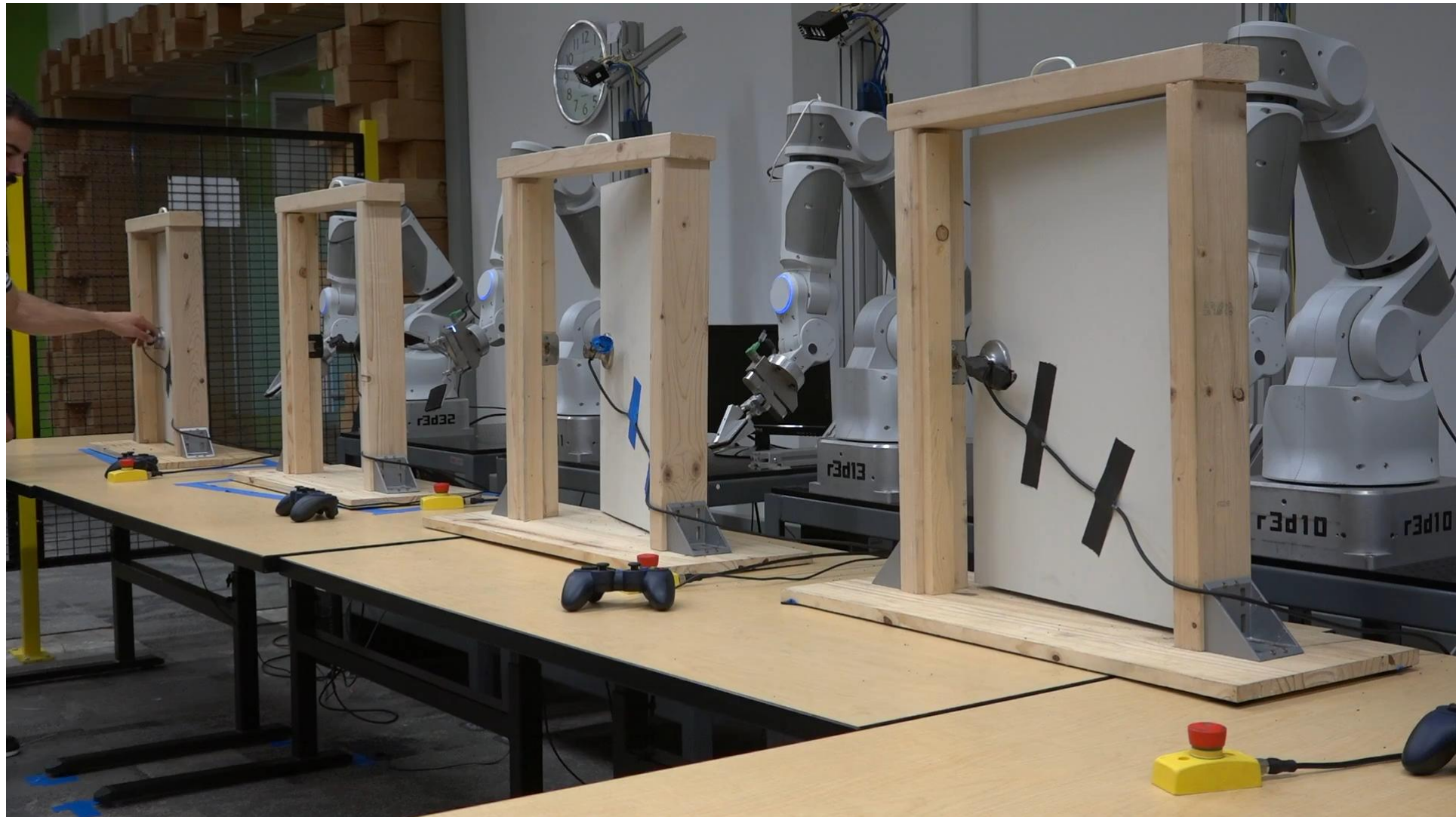
```
# Criar e treinar o modelo PPO
model = PPO('MlpPolicy', env, learning_rate=0.001, verbose=1)
model.learn(total_timesteps=20000)
```

```
# Testar o modelo treinado
obs = env.reset()
for _ in range(10):
    action, _states = model.predict(obs, deterministic=True)
    obs, reward, done, info = env.step(action)
    env.render()
    if done:
        obs = env.reset()
```

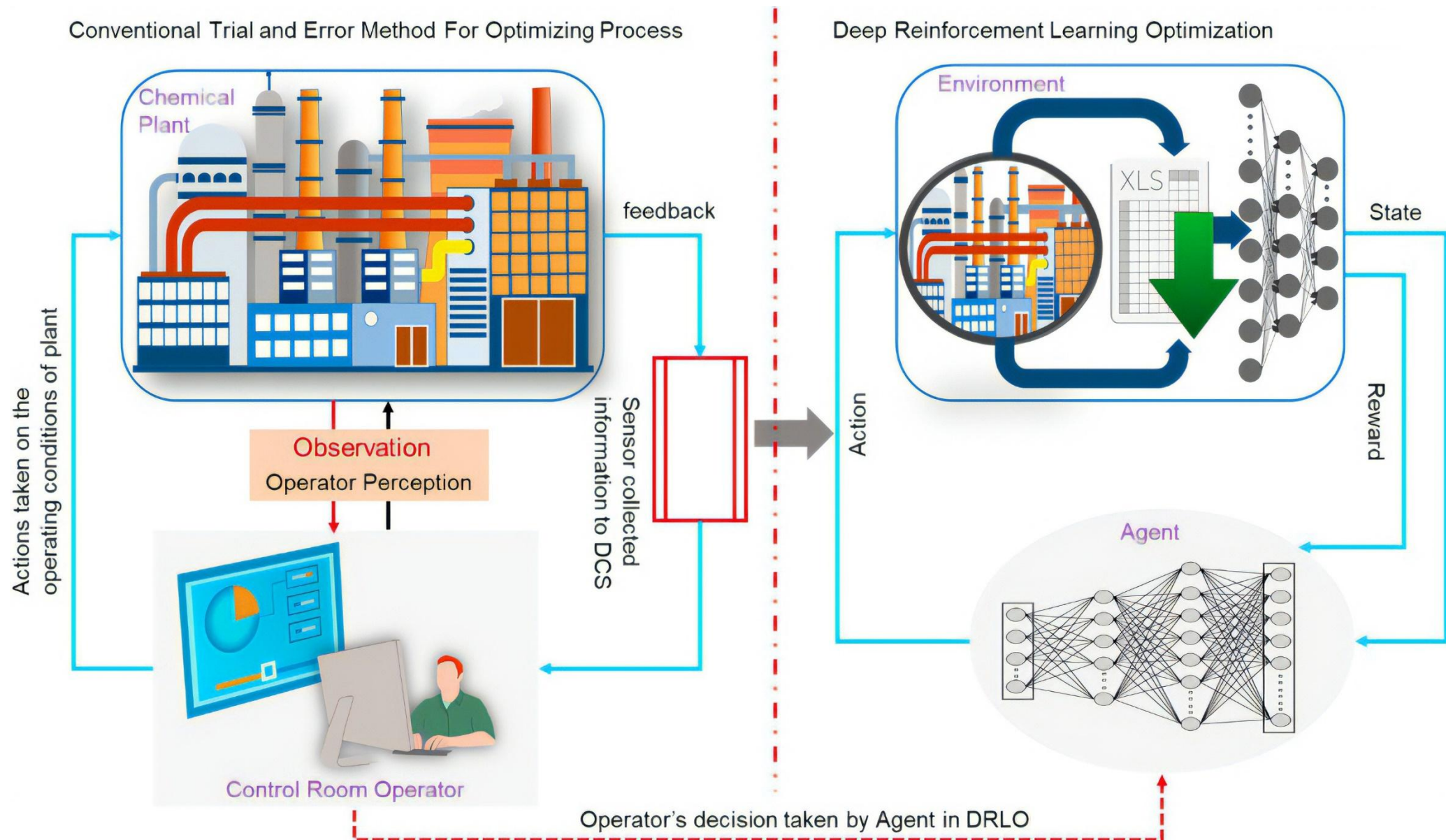
Bots para Jogos



Treino de Robots



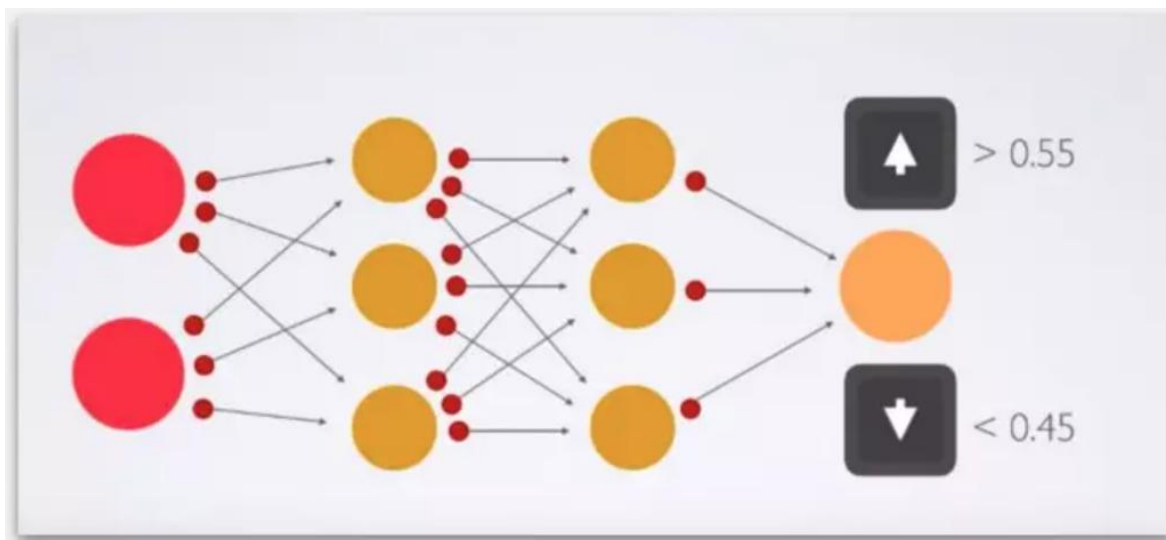
Otimização



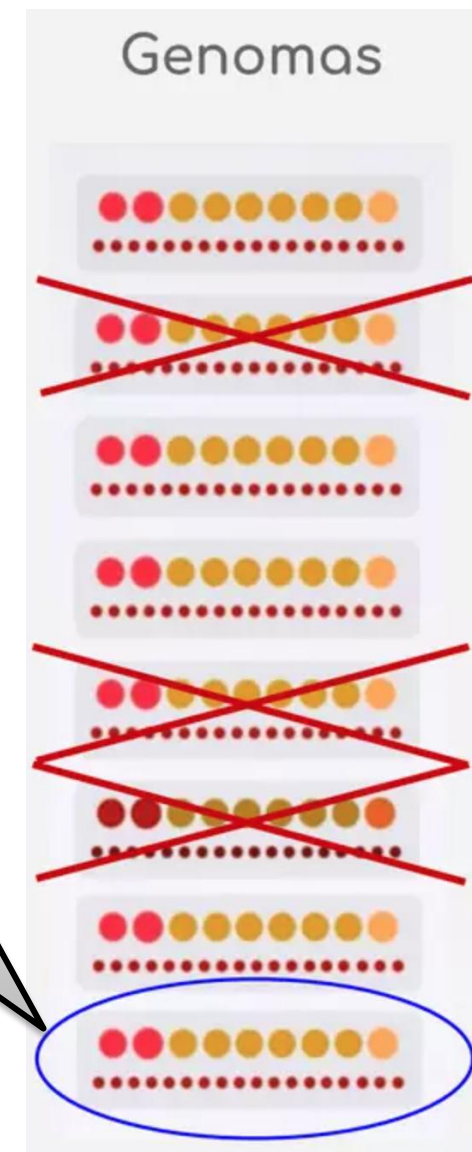
- ◆ Lab 08 - Tipos de aprendizagem - aprendizagem por reforço
 - 8.1 – Algoritmos de reforço

Algoritmo Genético

uma **geração** é um **ciclo** onde as soluções são criadas, avaliadas e evoluem



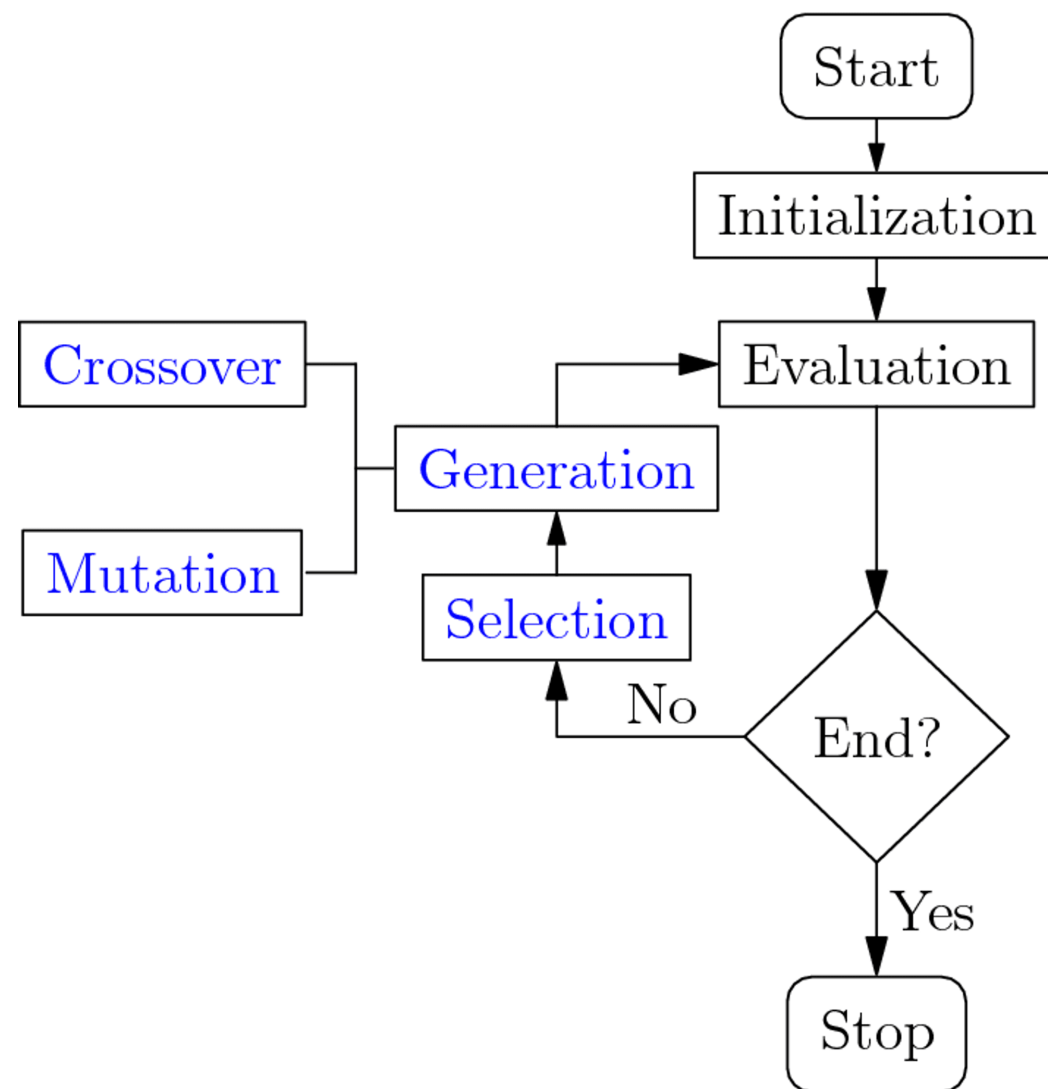
No final de cada geração, surge uma nova versão de **genomas**, resultante de cruzamentos e mutações, que são as próximas soluções a avaliar.



Algoritmo Genético

Um **Algoritmo Genético**, GA, é uma técnica de aprendizagem automática modelada na evolução biológica.

- ♦ **Seleção**: indivíduos são escolhidos para reprodução com base na sua aptidão.
- ♦ **Cruzamento**: Combina informações genéticas de dois progenitores para criar descendentes.
- ♦ **Mutação**: Introduz pequenas alterações aleatórias na genética dos descendentes.
- ♦ **Avaliação**: A função de aptidão avalia o desempenho de cada indivíduo na população



Aprendizagem Global e Aprendizagem Local

- ♦ **Aprendizagem Global:** Busca novas soluções em todo o espaço de possibilidades.
 - Imagine que está à procura de um bom restaurante numa cidade que não conhece.
- ♦ **Aprendizagem Local:** Melhora uma solução específica, ajustando-a dentro de uma pequena região.
 - Agora que encontrou um restaurante que parece bom, decide entrar e experimentar diferentes pratos do cardápio para escolher o seu favorito.

Comparar algoritmos genéticos

Algoritmo	Tipo	Prós	Contras
Algoritmo Genético Clássico	Baseado em Evolução	Encontra soluções globais; otimização adaptativa	Não é garantido encontrar a melhor solução; pode ser lento
NSGA-II	Evolução Multi-objetivo	Lida com múltiplos objetivos; não requer ponderação direta	Complexo; pode ser computacionalmente caro
Algoritmos Meméticos	Evolução e Aprendizagem Local	Combina busca global e local para melhorar soluções	Implementação complexa; alto custo computacional

Algoritmo Genético Clássico

```
# pip install deap

from deap import base, creator, tools, algorithms
import random

# Definir fitness e população
creator.create("FitnessMax", base.Fitness, weights=(1.0,))
creator.create("Individual", list, fitness=creator.FitnessMax)
toolbox = base.Toolbox()
toolbox.register("individual", tools.initRepeat, creator.Individual, random.randint, -10, 10, 1)
toolbox.register("population", tools.initRepeat, list, toolbox.individual)
toolbox.register("evaluate", lambda ind: (ind[0]**2,))
toolbox.register("mate", tools.cxBlend, alpha=0.5)
toolbox.register("mutate", tools.mutUniformInt, low=-1, up=1, indpb=0.2)
toolbox.register("select", tools.selTournament, tournsize=3)

# Executar o algoritmo genético
populacao = toolbox.population(n=10)
algorithms.eaSimple(populacao, toolbox, cxpb=0.5, mutpb=0.2, ngen=50, verbose=False)

# Melhor solução encontrada
melhor_individuo = tools.selBest(populacao, 1)[0]
print(f"Melhor solução: {melhor_individuo}, Fitness: {melhor_individuo.fitness.values[0]}")
```

NSGA-II

```
# pip install deap
```

```
from deap import base, creator, tools, algorithms
import random
```

```
creator.create("FitnessMulti", base.Fitness, weights=(1.0, 1.0))
creator.create("Individual", list, fitness=creator.FitnessMulti)
```

```
toolbox = base.Toolbox()
toolbox.register("individual", tools.initRepeat, creator.Individual, random.uniform, -10, 10, 2)
toolbox.register("population", tools.initRepeat, list, toolbox.individual)
toolbox.register("evaluate", lambda ind: (ind[0]**2, (ind[1] - 5)**2))
toolbox.register("mate", tools.cxBlend, alpha=0.5)
toolbox.register("mutate", tools.mutGaussian, mu=0, sigma=1, indpb=0.2)
toolbox.register("select", tools.selNSGA2)
```

```
pop = toolbox.population(n=100)
algorithms.eaMuPlusLambda(pop, toolbox, mu=100, lambda_=200, cxpb=0.7, mutpb=0.3, ngen=50,
verbose=False)
```

```
# Mostrar as melhores soluções
```

```
for ind in tools.sortNondominated(pop, len(pop), first_front_only=True)[0]:
    print(f"Indivíduo: {ind}, Fitness: {ind.fitness.values}")
```

Algoritmos Meméticos

```
# pip install deap
```

```
from deap import base, creator, tools, algorithms
import random
```

```
creator.create("FitnessMax", base.Fitness, weights=(1.0,))
creator.create("Individual", list, fitness=creator.FitnessMax)
```

```
toolbox = base.Toolbox()
toolbox.register("individual", tools.initRepeat, creator.Individual, random.uniform, -10, 10, 1)
toolbox.register("population", tools.initRepeat, list, toolbox.individual)
toolbox.register("evaluate", lambda ind: (ind[0]**2,))
toolbox.register("mate", tools.cxBlend, alpha=0.5)
toolbox.register("mutate", tools.mutGaussian, mu=0, sigma=1, indpb=0.2)
toolbox.register("select", tools.selTournament, tournsize=3)
```

```
# Função de busca local (hill climbing simples)
```

```
def local_search(individual):
    for _ in range(5): # 5 iterações de melhoria
        new_value = individual[0] + random.uniform(-0.5, 0.5)
        if new_value**2 > individual[0]**2:
            individual[0] = new_value
    return individual
```

Algoritmos Meméticos

```
# Executar o algoritmo memético
pop = toolbox.population(n=10)
for _ in range(50): # 50 gerações
    offspring = algorithms.varAnd(pop, toolbox, cxpb=0.5, mutpb=0.2)
    for ind in offspring:
        local_search(ind) # Aplicar busca local
    pop = toolbox.select(offspring + pop, k=len(pop))

# Melhor solução encontrada
best = tools.selBest(pop, 1)[0]
print(f"Melhor solução: {best}, Fitness: {best.fitness.values[0]}")
```

- ◆ Lab 08 - Tipos de aprendizagem - aprendizagem por reforço
 - 8.2 – Algoritmos Genéticos